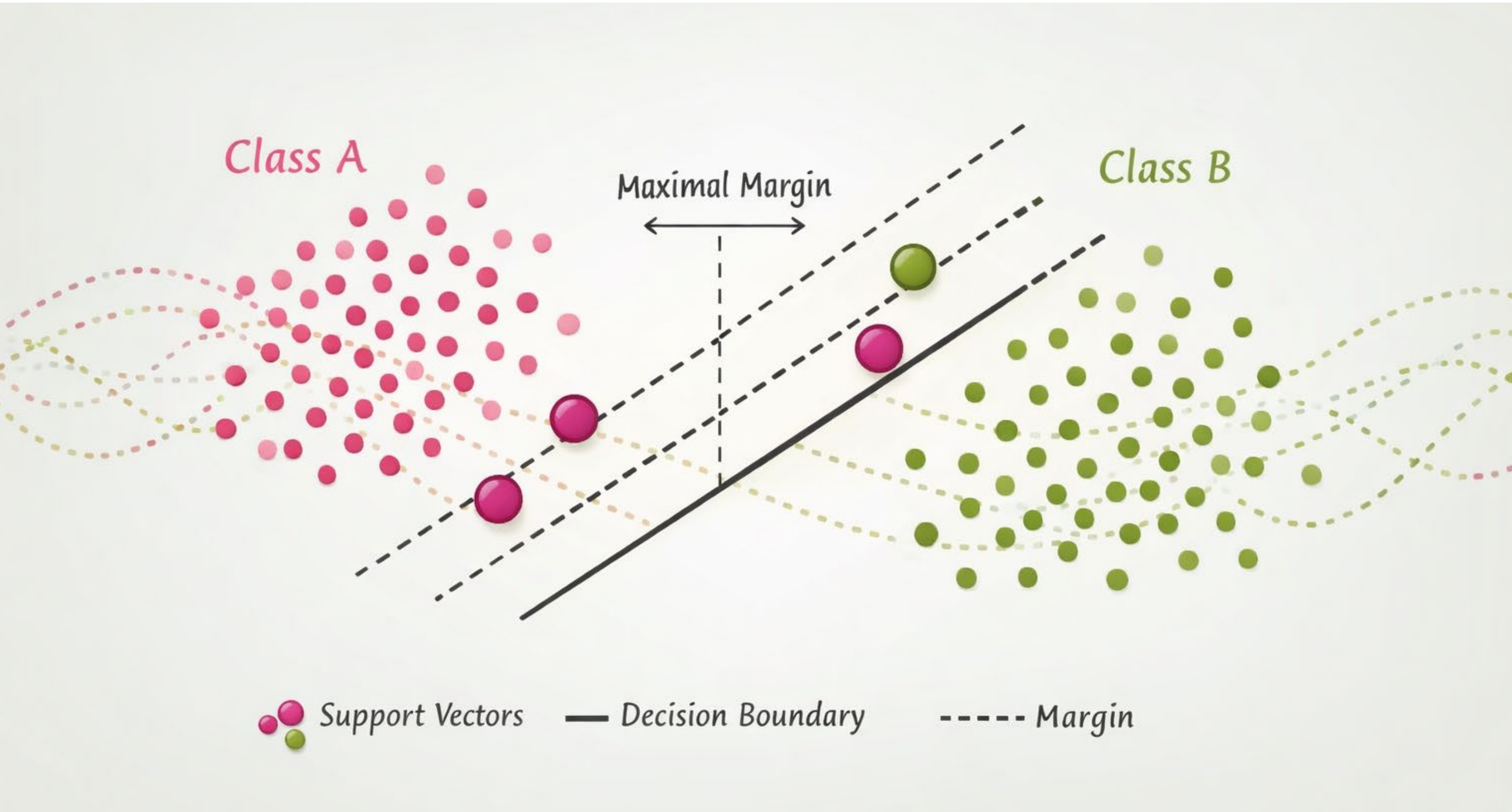
Support Vector Machines



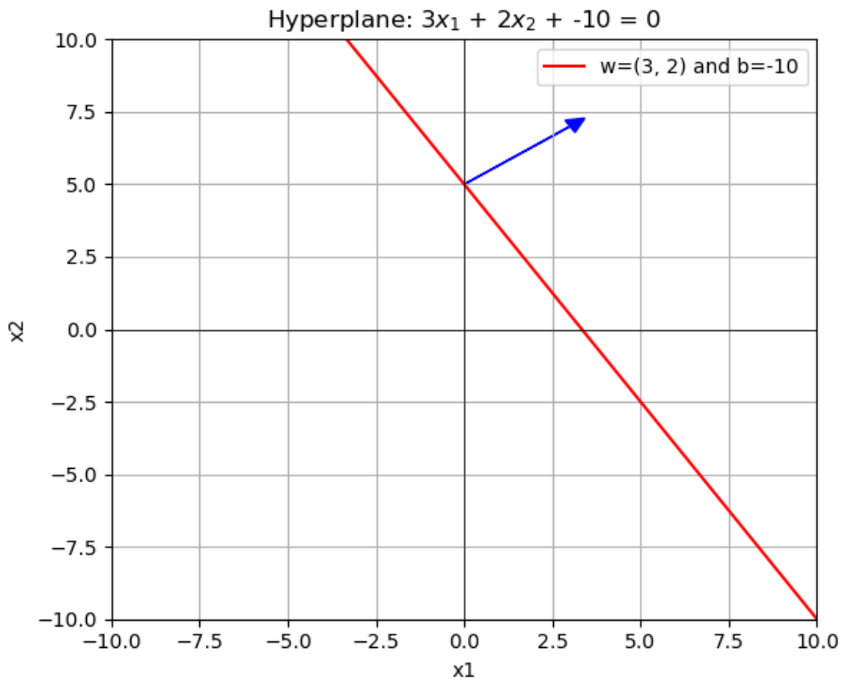
Introduction to SVM

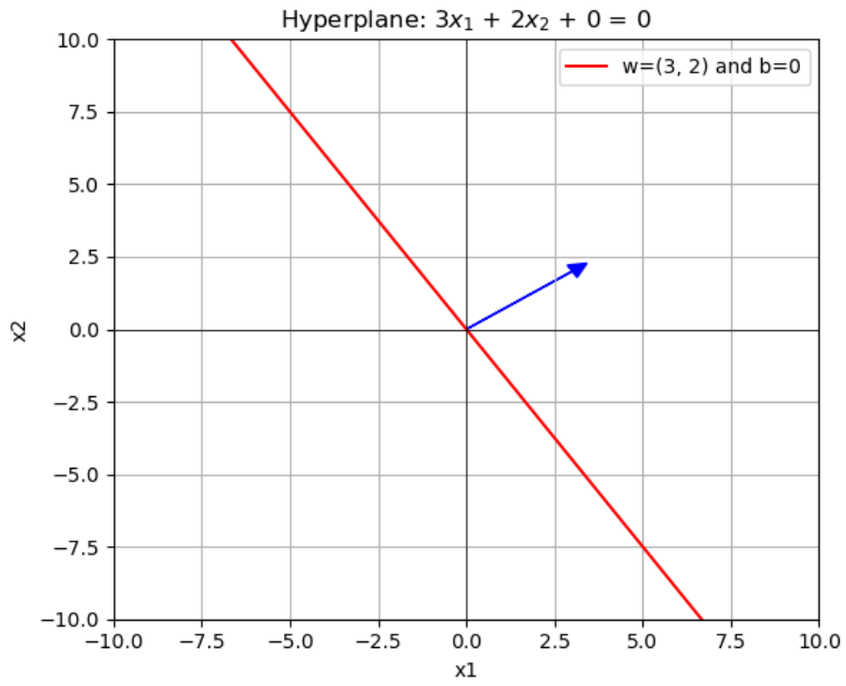
- The support vector machines (SVM) is an approach developed in the 1990s by computer scientists
  - (No probability estimate) X
- Often work well for many problems types of problems
  - probably less used now with advances in ANN
  - but still kind of cool mathematically
- First we need to review the definition of a hyperplane

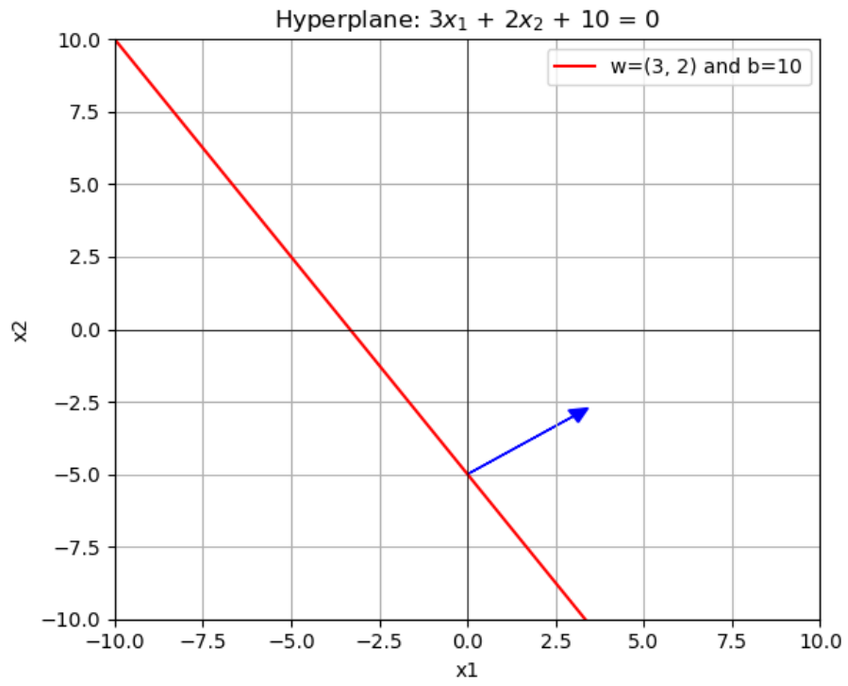
Hyperplanes

- A hyperplane is a generalization of a line in p dimensions
- Formally, a hyperplane in p dimensions is a flat affine subspace of dimension $p - 1$ with general form
  - A hyperplane in N-dimensional space is defined as
$$\mathbf{w} \cdot \mathbf{x} + b = 0$$
    - Where $\mathbf{w}$ is the weight vector (perpendicular to the hyperplane) that determines the orientation
    - b is the bias term
    - $\mathbf{x}$ is any point in the space
- When $p = 2$ (there are two features) a hyperplane is a line

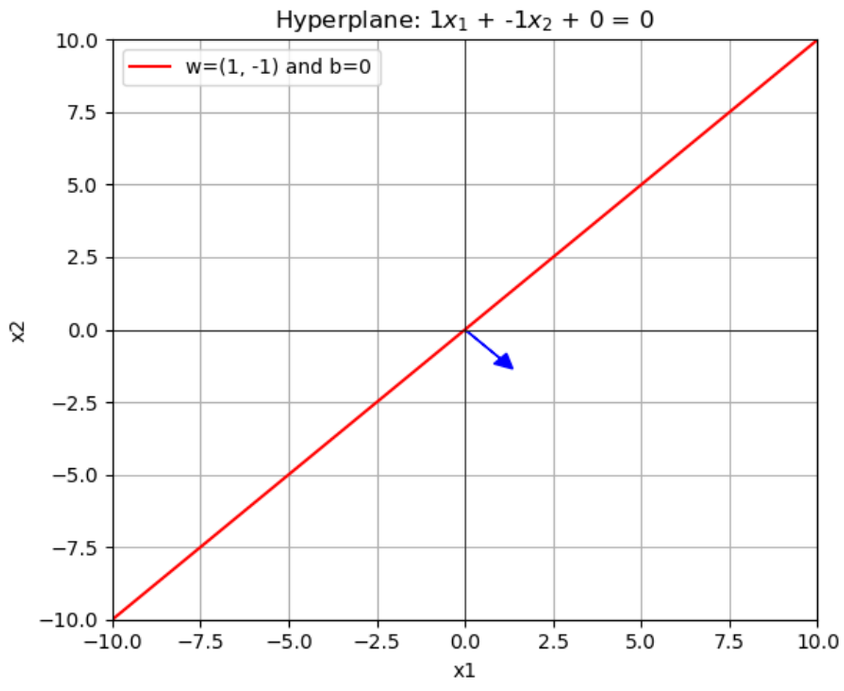
Changing b

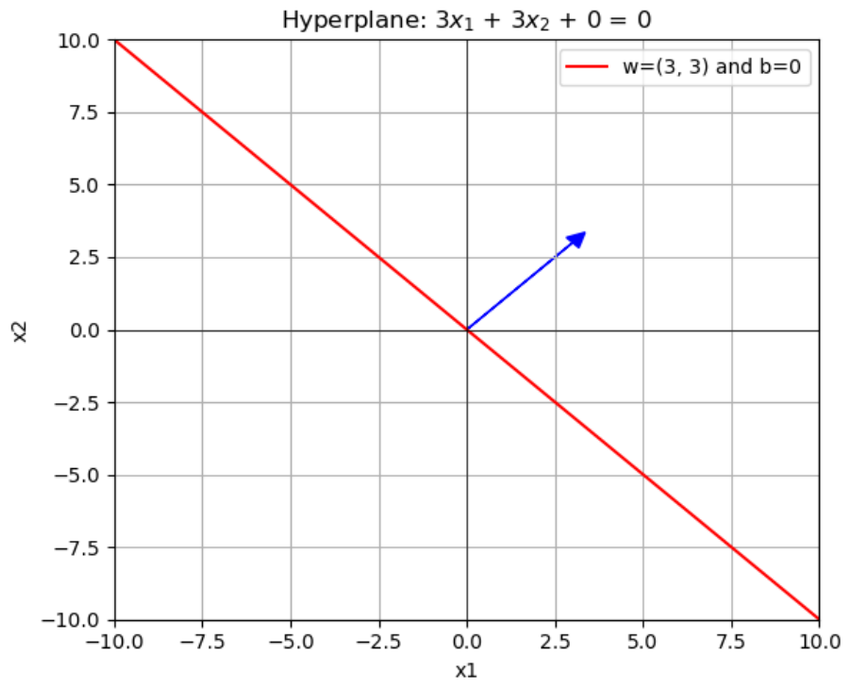


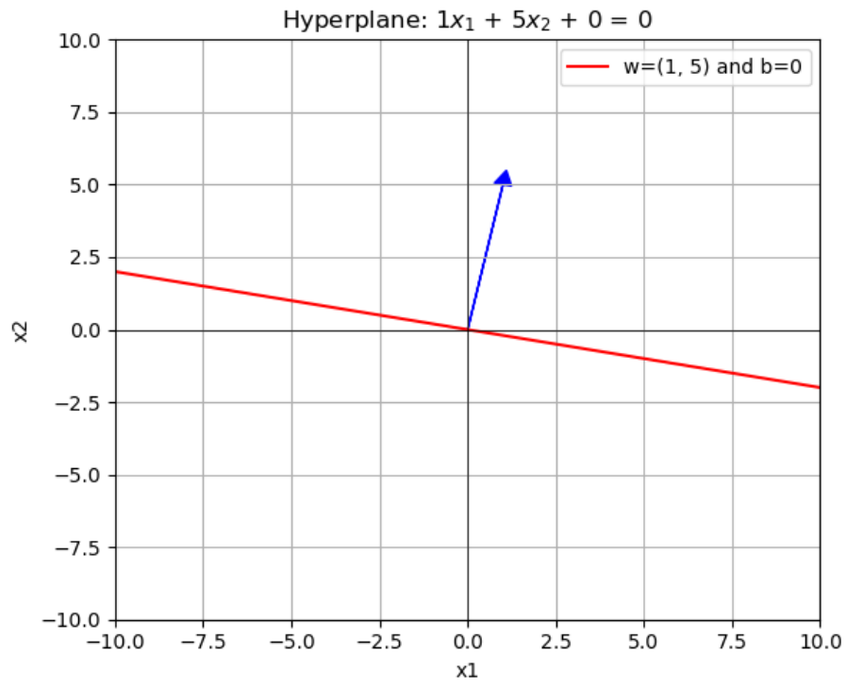




Changing $\mathbf{w}$







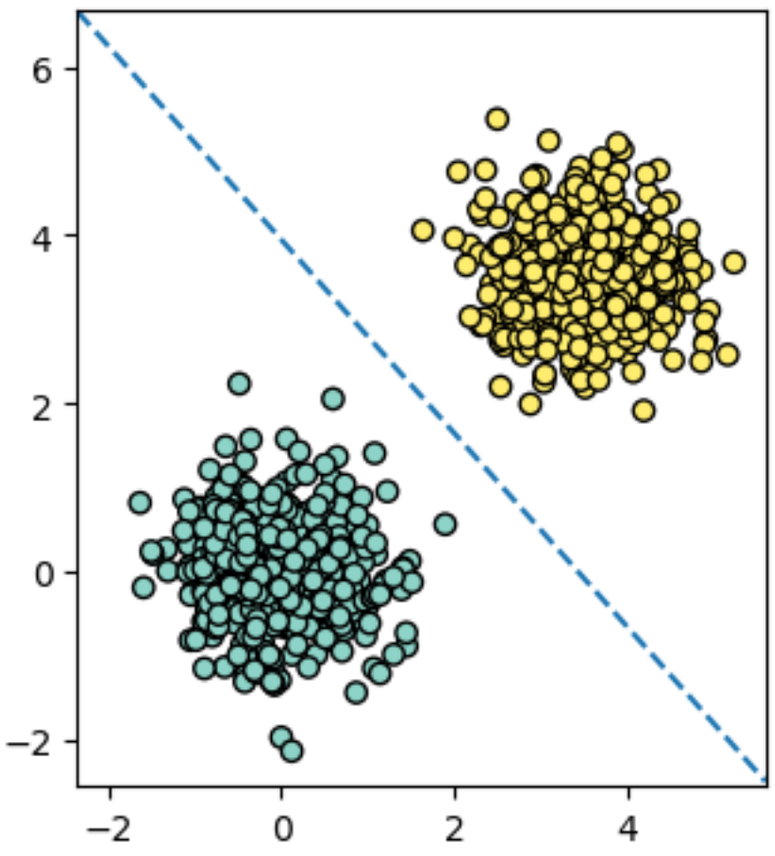
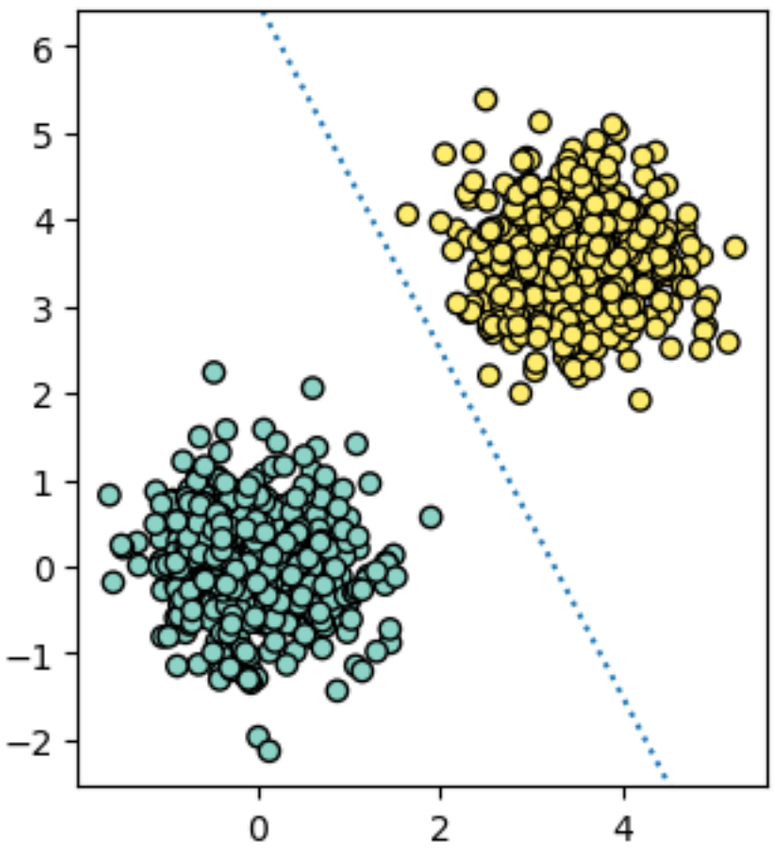
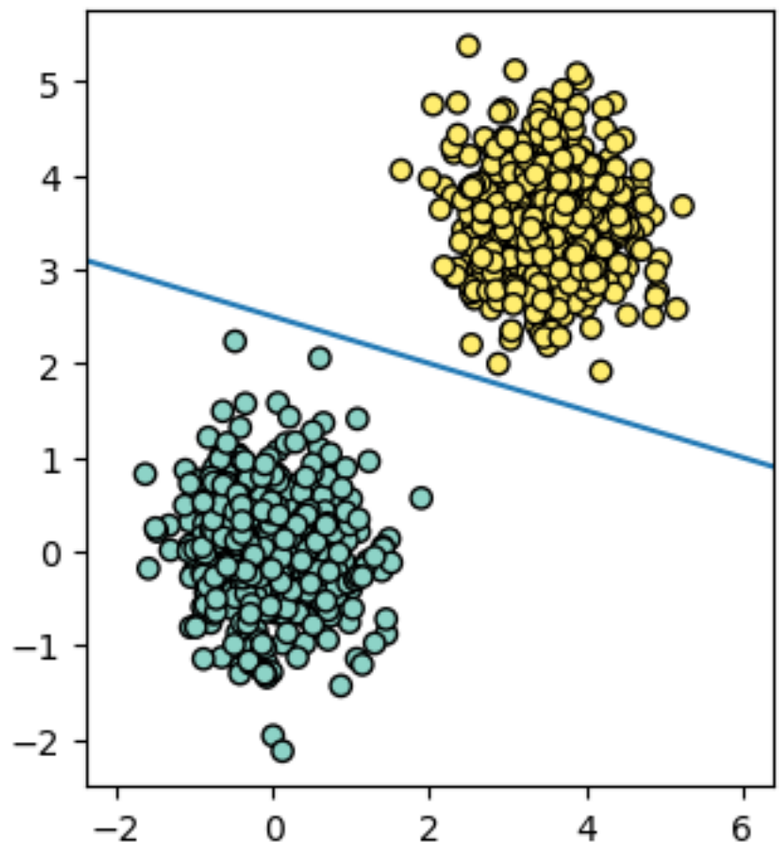
High-level view of SVM

Introduction to SVM

- The main idea:
  - Find the hyperplane that **completely** and **optimally** separates two classes in a feature space

Separate the group

- Consider two linearly separable groups
  - groups can be separated with a hyperplane
- Which line (hyperplane) is the “best” separator?

Introduction to SVM

- The main idea:
  - Find the hyperplane that **completely** and **optimally** separates two classes in a feature space

Introduction to SVM

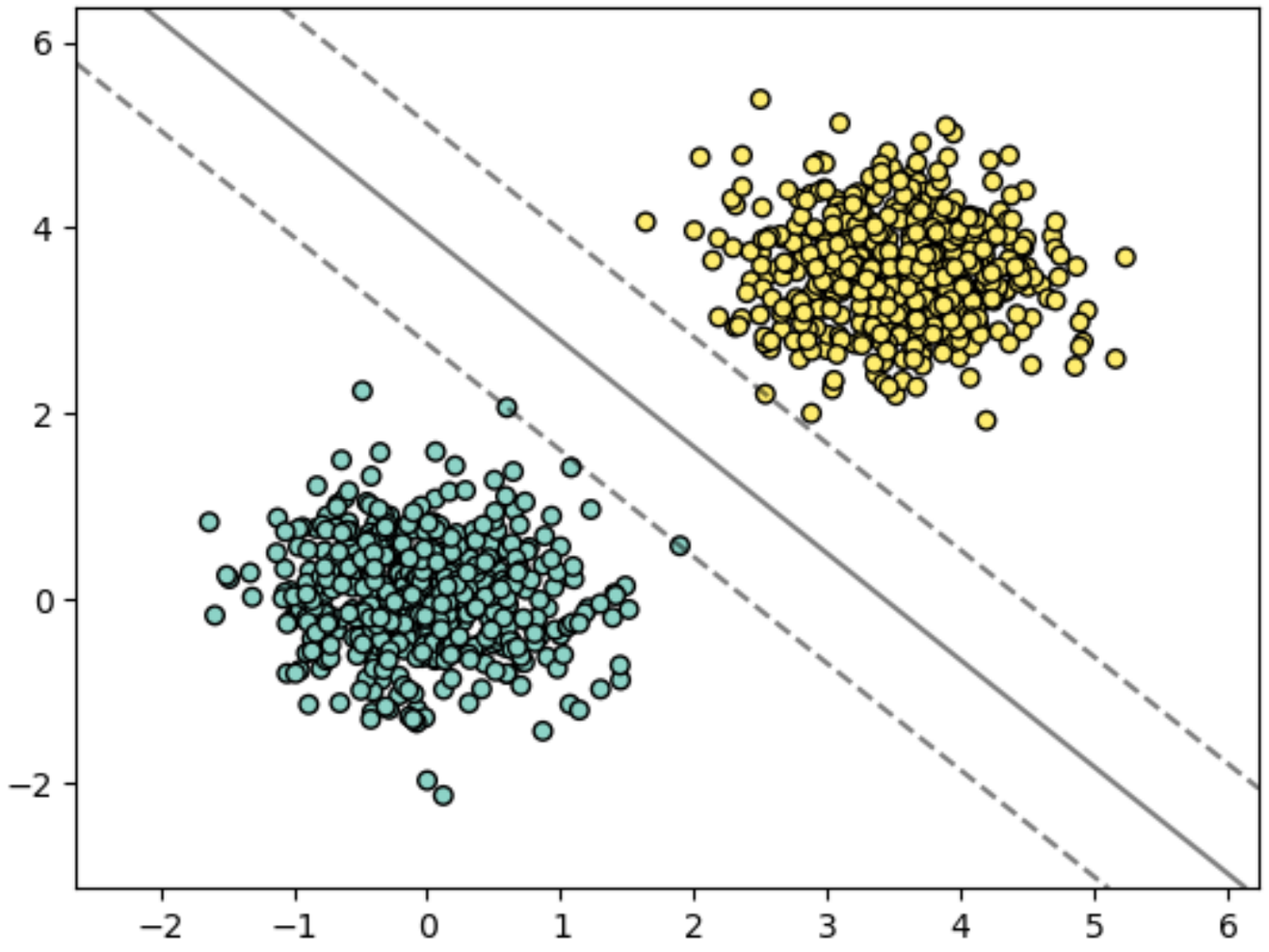
- The main idea:
  - Find the hyperplane that completely and optimally separates two classes in a feature space
  - If there is **no such hyperplane** (because classes aren't linearly separably)

    1. Soften the definition of "completely separates"
    2. Expand the feature space so that separation is possible

Maximal margin (or hard margin) classifier

Maximal Margin Classifier

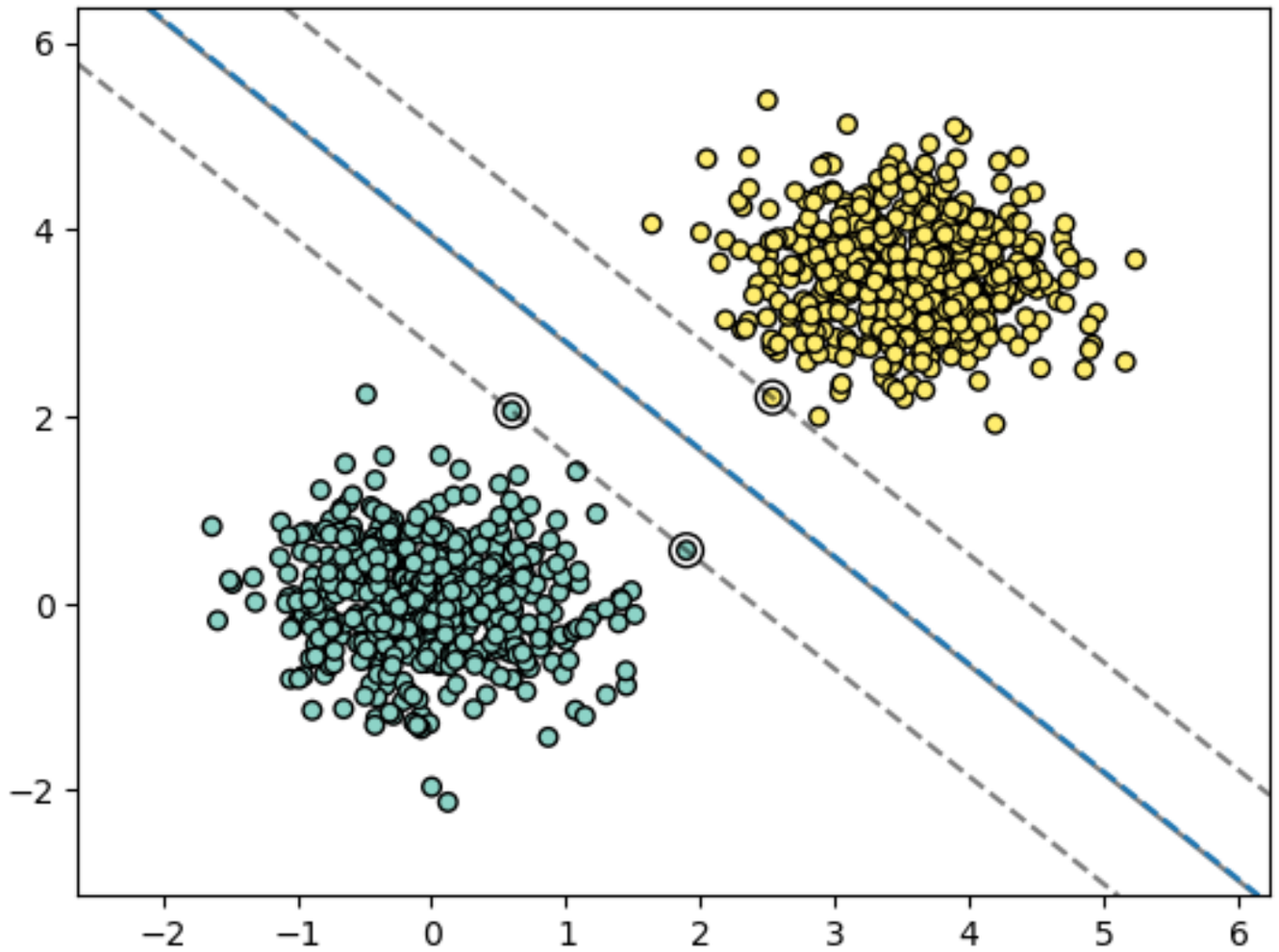
- Decision boundary is the hyperplane that is farthest from the training observations
- Fits the “widest possible street”



Maximal Margin Classifier

- Notice that only the points on the edge of the street matter
- These are called support vectors (decision boundary is supported by these instances)

The support vectors are just points in this two dimensional feature space, but in higher dimensions, they will be actual vectors.



Margins

- The decision boundary margin is the distance between the hyperplane and the nearest data point(s) from either class

- For a given point $\mathbf{x_i}$ with label y_i (+1 or -1) the distance to the margin can be computed by

$$\frac{y_i(\mathbf{w} \cdot \mathbf{x_i} + b)}{||w||}$$

- We want to find the largest margin by maximizing

$\frac{1}{||\mathbf{w}||}$ which is equivalent to minimizing $\frac{1}{2}||\mathbf{w}||^2$ or $\frac{1}{2}\mathbf{w}'\mathbf{w}$

Hinge Loss

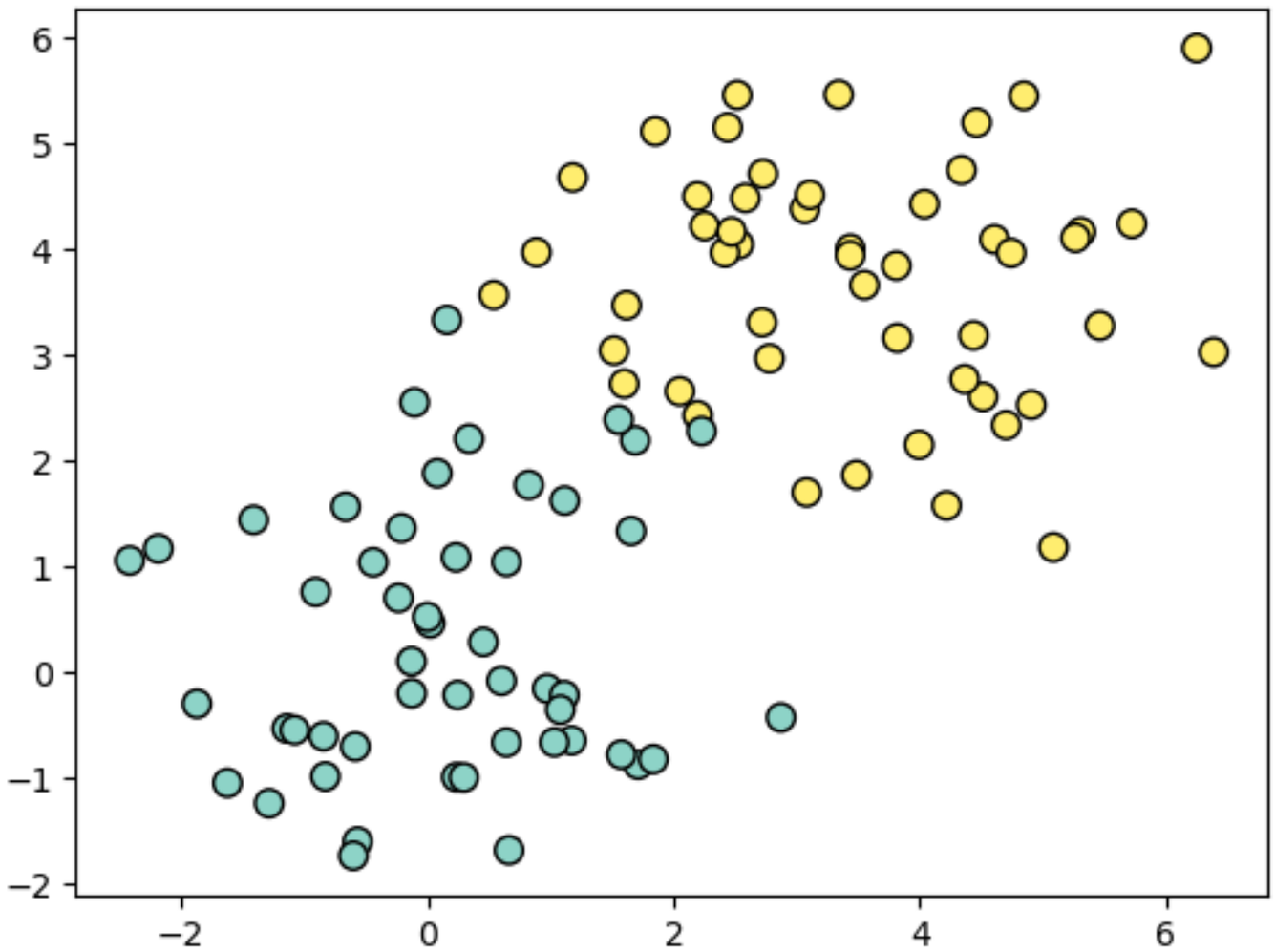
- Sometimes you'll see the term "hinge loss" in connection with SVM
- For and individual data point x_i with label y_i, the hinge loss is

$$loss(\mathbf{x_i}, y_i, \mathbf{w}, b) = max(0, 1 - y_i(\mathbf{w} \cdot \mathbf{x_i} + b))$$

- When summed across all data points we get the hard margin cost function

Potential problems

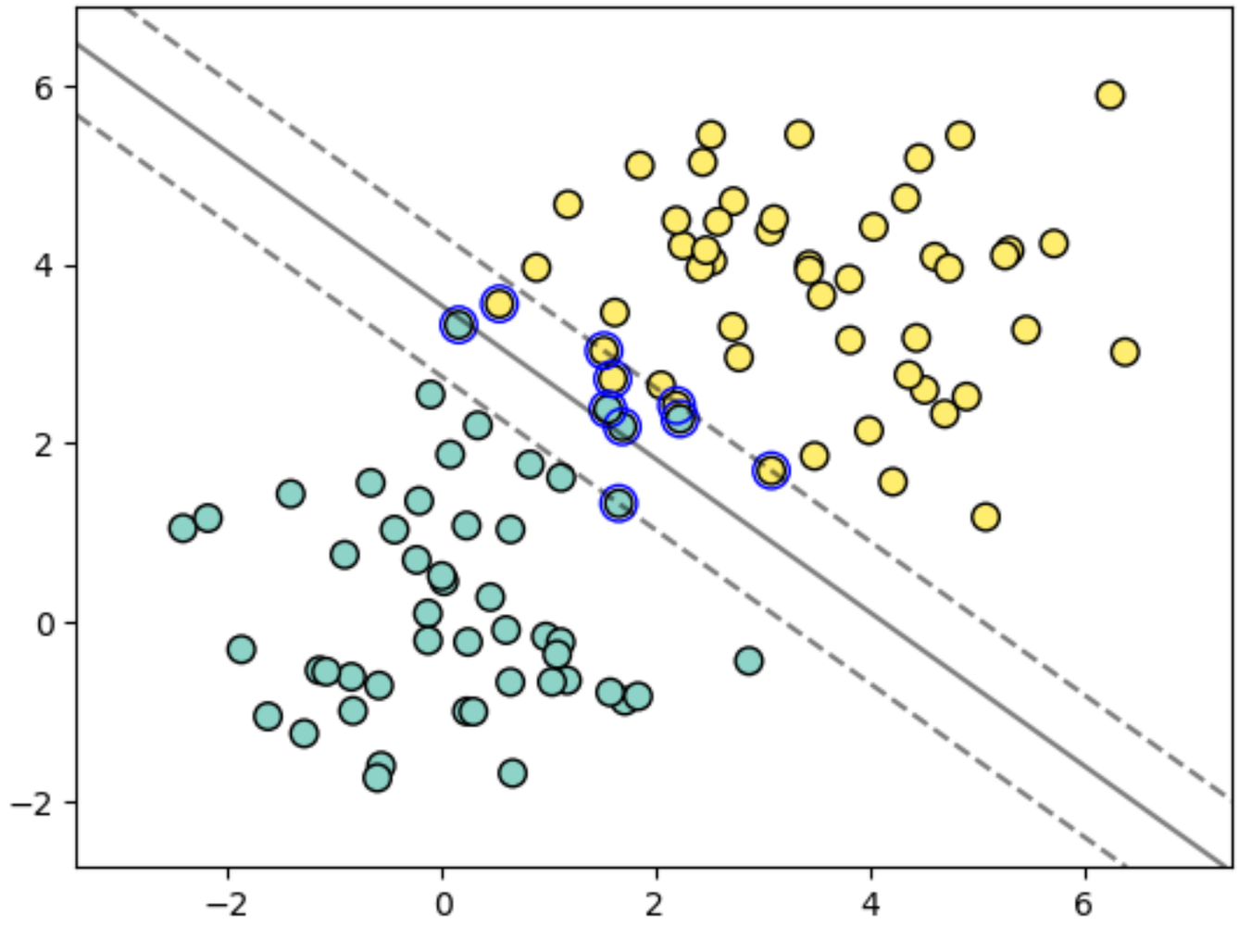
- What if a separating hyperplane does not exist?
- Then there is no maximum marginal classifier



Soft Margin SVM

Potential problems

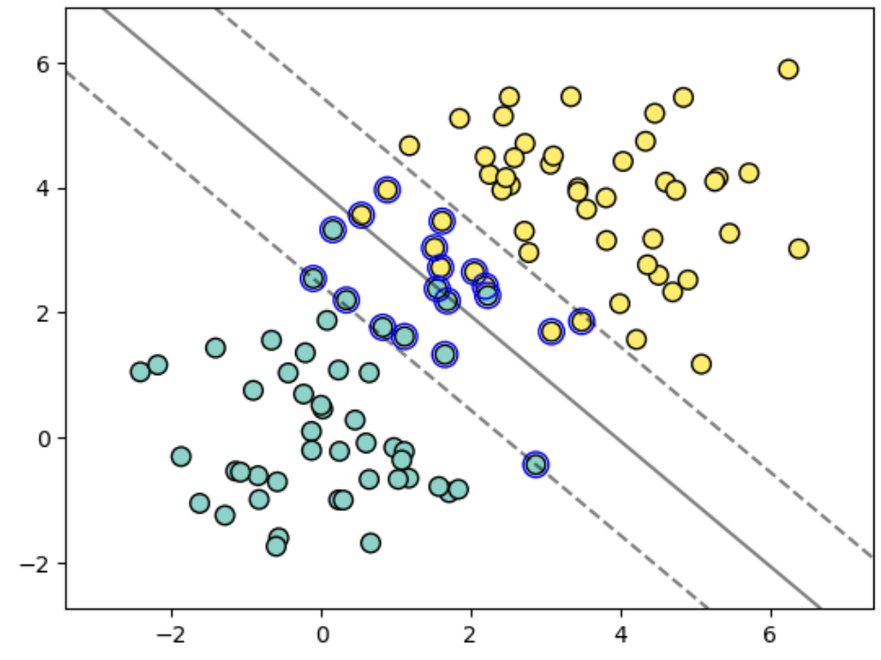
- In this case, a natural generalization would be to allow some points to spill into the “street” or margin



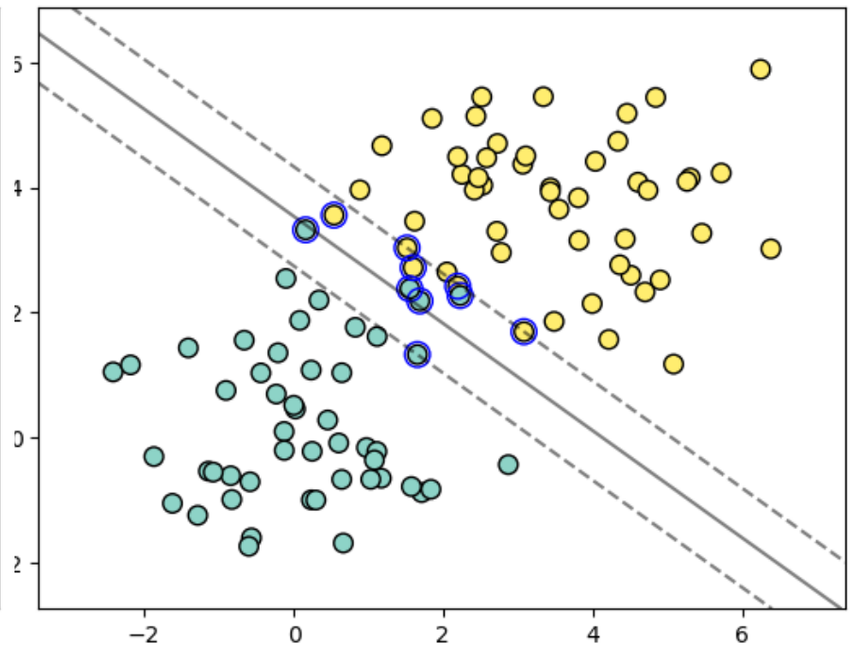
Support Vector Classifier (Soft margin)

- Hyper-parameter **C** controls the width of the road and the amount of over-spill that is allowed
- When **C** is small, the margin is wider and more points are involved in determining the direction of the boundary

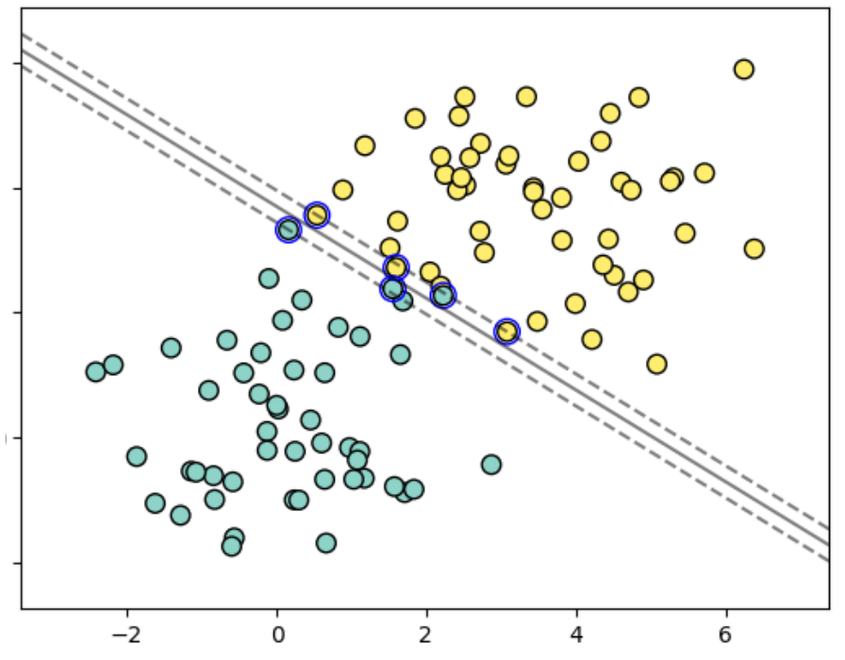
C=0.1



C=1



C=10



Slack Variables

- In soft margin SVM, we introduce slack variables, ξ_i
- ξ_i measures the degree to which a particular data point violates the margin constraint
- For each $\mathbf{x_i}$
  - $\xi_i = 0$: If x_i is correctly classified and outside the margin
  - $0 < \xi_i < 1$: If x_i lies inside the margin but is correctly classified
  - $\xi_i \geq 1$: If x_i is misclassified

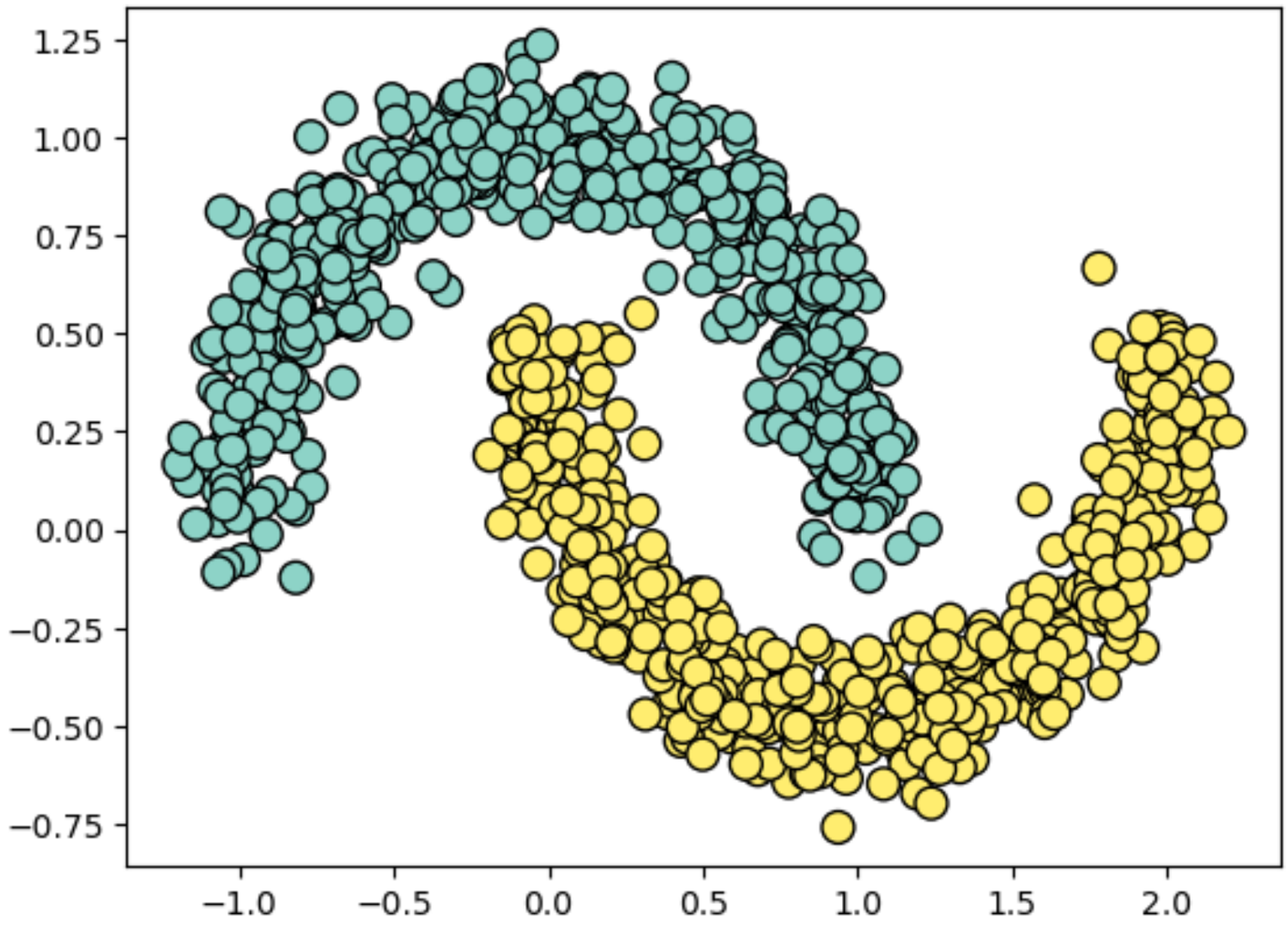
Support Vectors

- The points inside the margin (or points wrongly misclassified, if any) are the support vectors
  - **Only these points effect the decision boundary (the support vectors influence the position and orientation of the hyperplane)**
  - When C is small, the street is wider so more points are involved in making the decision boundary
    - Can lead to underfitting
  - When C is large, the street more narrow so fewer points are involved in making the decision boundary
    - Can lead to overfitting

Non-linearly separable data

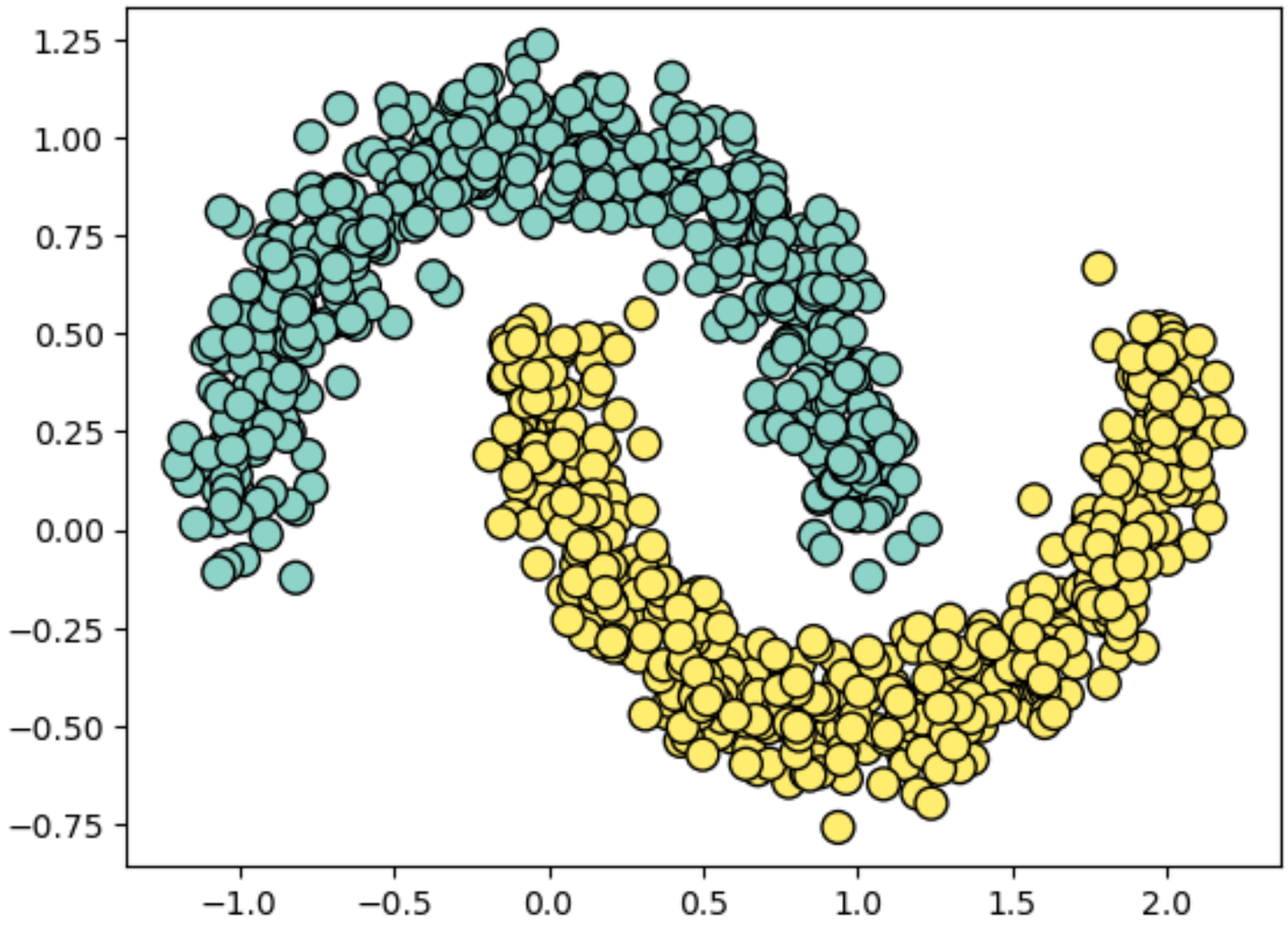
Non-linearly Separable

- Now what happens if the classes aren't linearly separable



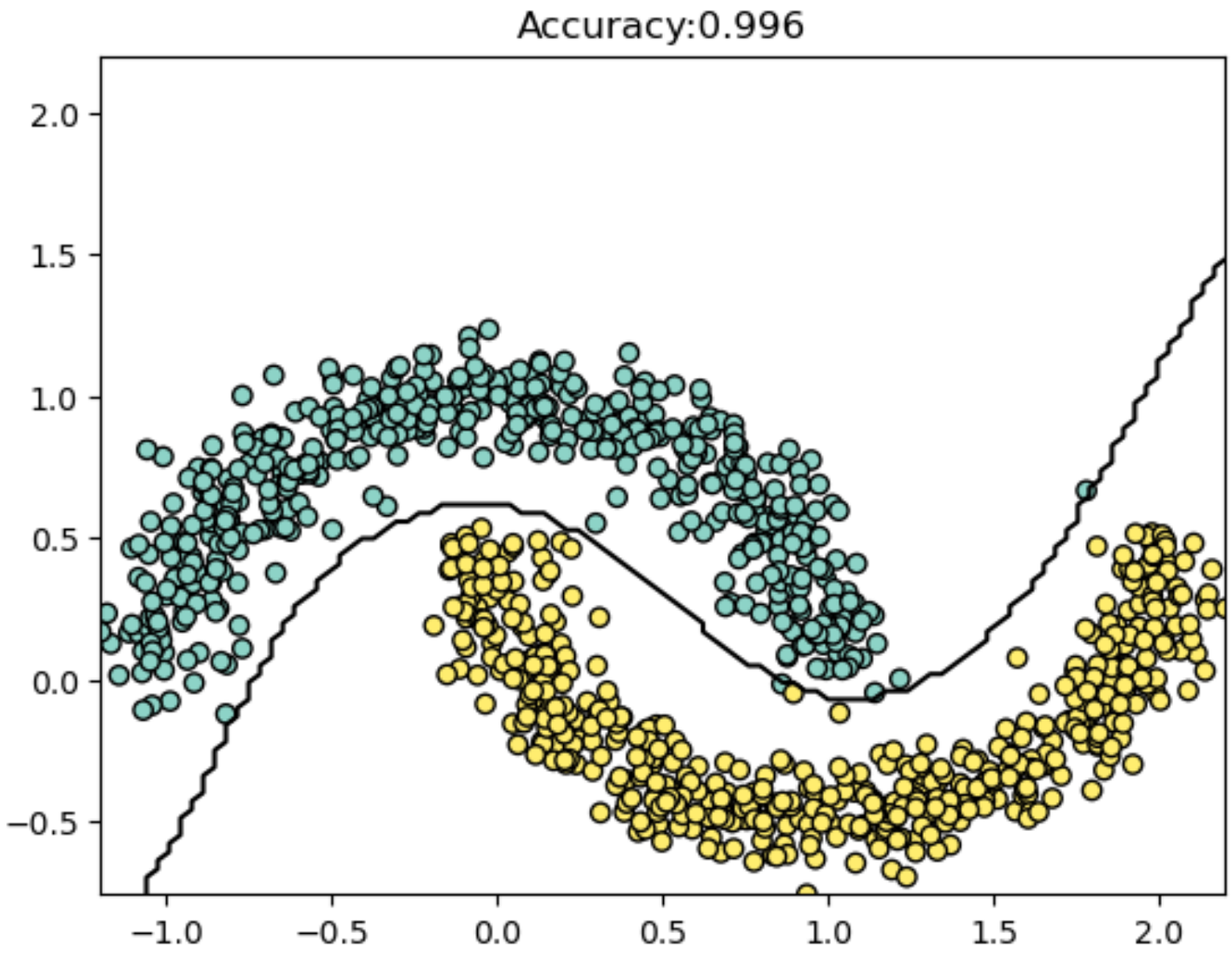
Non-linearly Separable

- One possibility is to add polynomials terms to a linear model (e.g. logistic regression)



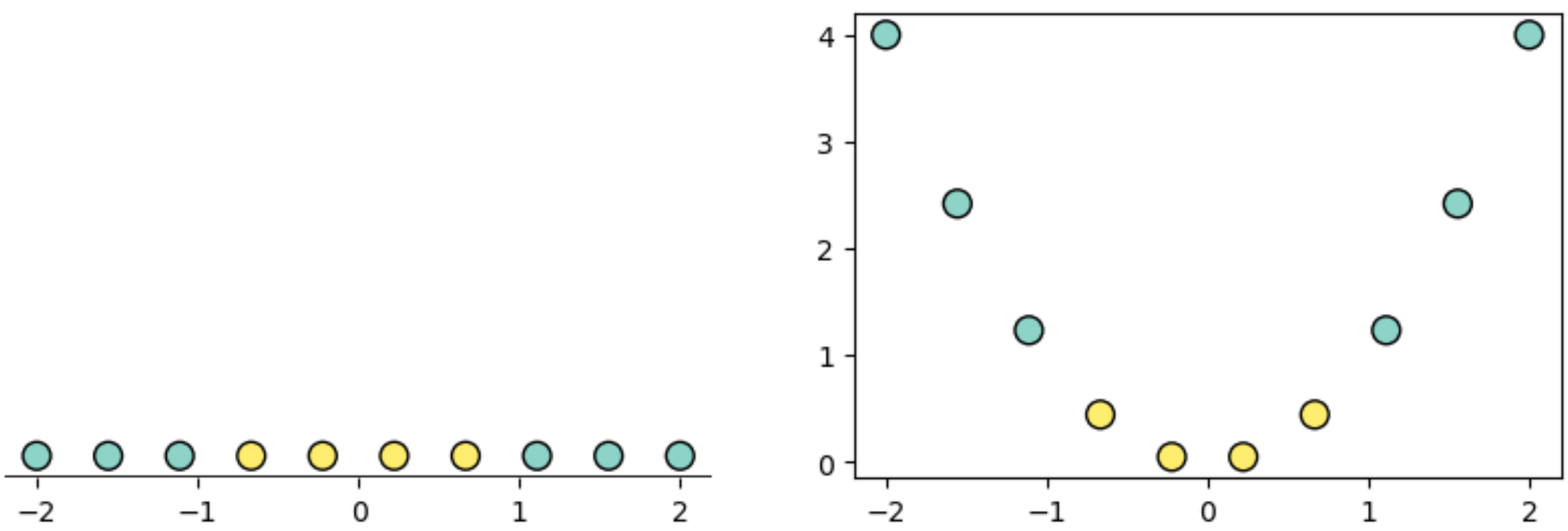
Non-linearly Separable

- This is the decision boundary of a logistic regression model after adding 4th order polynomials to the feature matrix



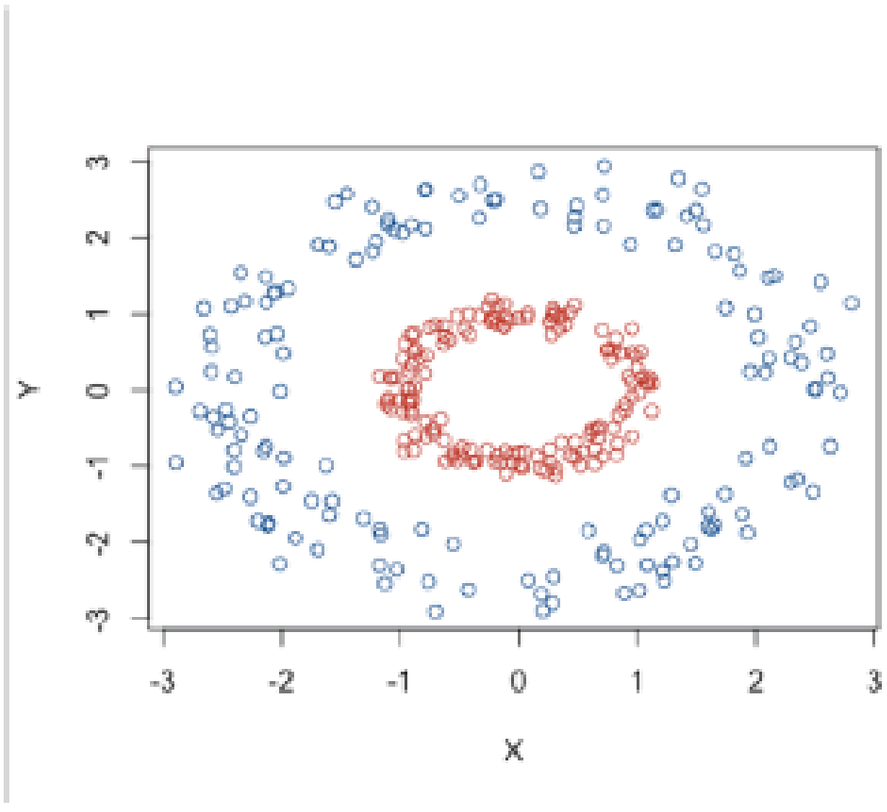
Examples

- Here we can visualize how adding higher dimensions can create linear separability
  - In 1-dimension the data is not linearly separable
  - After applying the transformation $\phi(x) = x^2$ we have linear separation

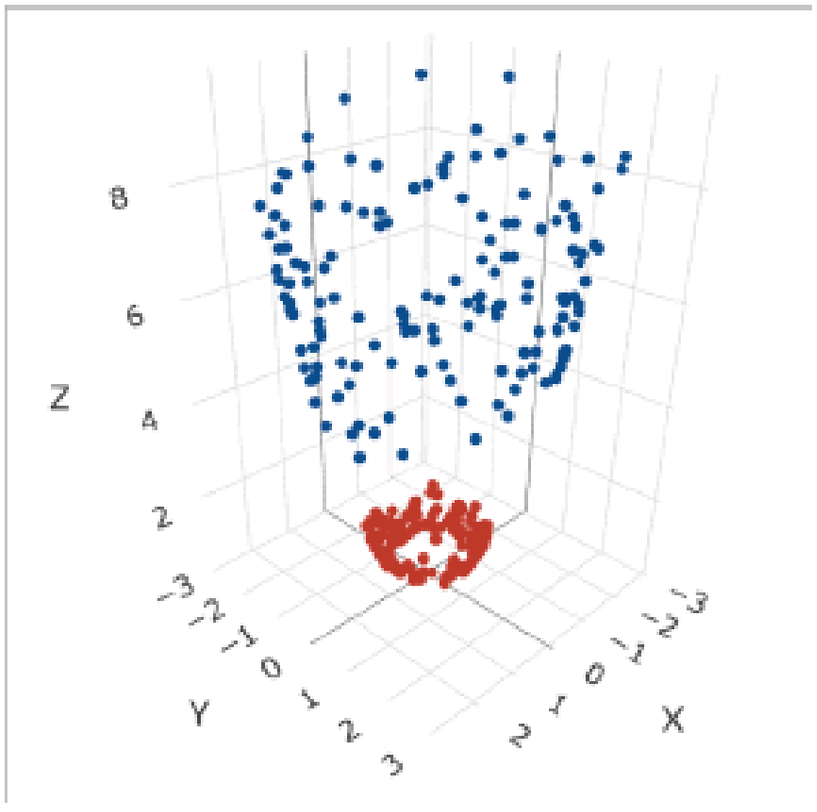


Another Example

Another Example



(a) Prior to application of $\phi()$



(b) Following application of $\phi()$

Figure 5: The 'Kernel Trick': Transforming data to higher dimensions to gain linear separability [21].

A solution and a problem

- While mapping to higher-dimensional space can create linearly separable data…
- …It can be computationally expensive to directly compute the transformation
  - E.g., polynomial space gets really big really fast, especially with lots of features
- …It is also hard to determine how many higher-dimensions to include
- This is where SVMs and the **Kernel Trick** come to the rescue

Support Vector Machines

Support Vector Machines

- The main idea behind SV*Machines* is to
  - create linear separation by expanding the feature space
  - but in a specific way using **kernels**
    - a computationally efficient trick for expanding the feature space
  - this is called the **kernel trick**

Dot Products

- Given two vectors in n-dimensional space:

  $\mathbf{a} = (a_1, a_2, \ldots, a_n)$ and $\mathbf{b} = (b_1, b_2, \ldots, b_n)$

- The dot product between $\mathbf{a}$ and $\mathbf{b}$ is

$$\langle \mathbf{a}, \mathbf{b} \rangle = \sum_{i=1}^{n} a_i b_i = \mathbf{a}'\mathbf{b}$$

Results from previous slide

- The dot product of the transformed vectors is equal square of the dot product of the original vectors

- Or in other words:

$$\phi(\mathbf{a})^{\mathsf{T}}\phi(\mathbf{b}) = (\mathbf{a}'\mathbf{b})^2$$

- In machine learning, a **kernel function, $K(\mathbf{a}, \mathbf{b})$,** is a function capable of computing the dot product between two transformed vectors based only on the original vectors **without having to compute the transformation ϕ**

$$K(\mathbf{a}, \mathbf{b}) = \phi(\mathbf{a})^{\mathsf{T}}\phi(\mathbf{b})$$

Common Kernels

- The previous example uses the 2nd order polynomial kernel
- Common kernels:
  - Linear: $K(\mathbf{a}, \mathbf{b}) = \mathbf{a}'\mathbf{b}$
  - Polynomial: $K(\mathbf{a}, \mathbf{b}) = (\gamma \mathbf{a}'\mathbf{b} + r)^d$
  - Gaussian RBF: $K(\mathbf{a}, \mathbf{b}) = \exp(-\gamma(||\mathbf{a} - \mathbf{b}||^2)$

So how does this all fit in to SVM?

- Primal vs Dual Problem
  - Given a constrained optimization problem (the primal problem) …
  - …It is possible to express a different but related problem, called the dual problem
  - The dual problem is usually faster to solve
  - But the dual problem typically gives a lower bound to the solution of the primal problem
  - Under some conditions, the primal and dual problems have the same solution and SVM meets these conditions

Math is cool

- If you've never taken an optimization class, the concept of primal vs dual can be a little confusing
- The dual formulation is basically the expression of the optimization problem in terms of Lagrange multipliers
- HOML give some of the formulas for going back and forth between the primal and dual problems
- Quote from Book:
  If you are starting to get a headache, that's perfectly normal: it's an unfortunate side effect of the kernel trick

Summary of the Kernel Trick in SVM

- A kernel function replaces the dot product in the SVM objective function
- By using the kernelized version
  - we implicitly map the input data into higher-dimensions…
  - …where it might be linearly separable
  - …without needing to perform the transformation explicitly

Gaussian RBF Kernel

More about the RBF Kernel

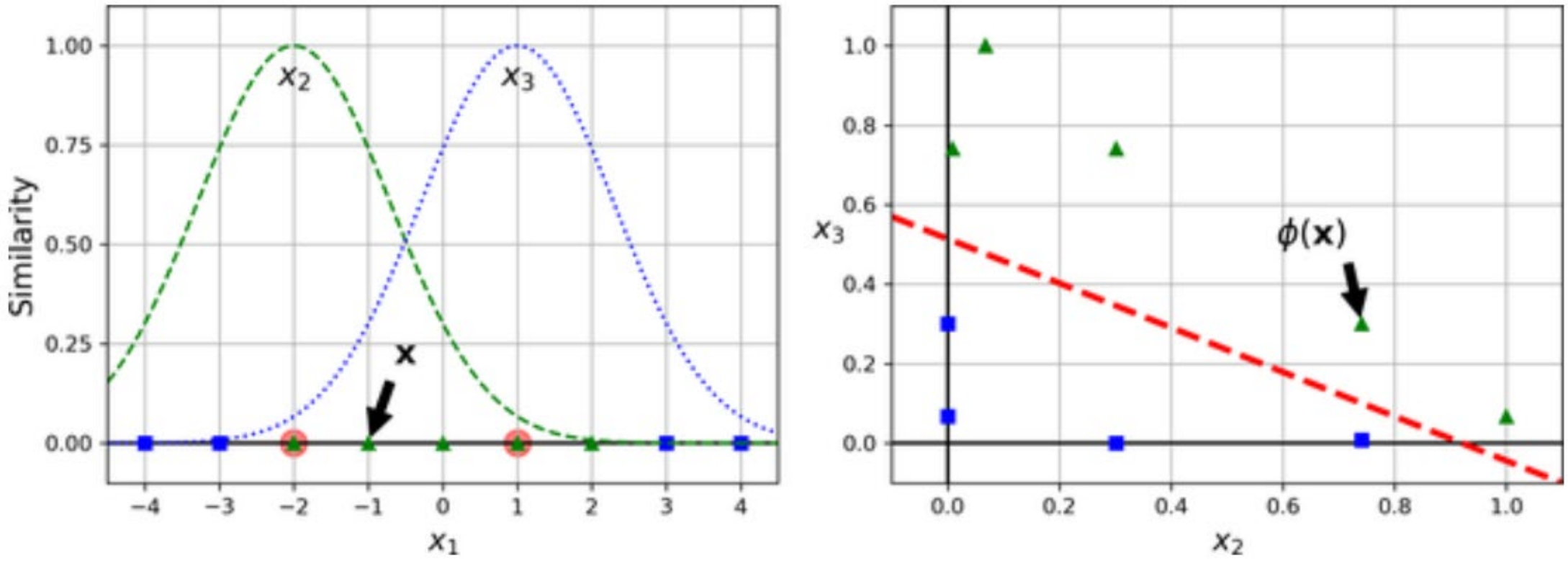
- The Gaussian RBF kernel is very popular and is the default for the **SVC** and **SVR** functions in sklearn

- $K(\mathbf{a}, \mathbf{b}) = \exp(-\gamma(||\mathbf{a} - \mathbf{b}||^2)$

More about the RBF Kernel

- Main idea
  - Rather than add polynomial features, we can also add “similarity” features
  - The “similarity” features will also transform to a higher dimension
    - Similarity features represent the similarity between two or more data points
    - For example we can add features based on the RBF similarity function from several arbitrary landmarks
    - RBF kernel corresponds to infinite-dimensional space

Example of Similarity Features



- Example where original features space is 1-dimensional (just x_1)
- Let x_2 be the similarity of x_1 to landmark 1 and x_3 be the similarity of x_1 to landmark 2
- For example, if $\gamma = 0.3$, for $x_1 = -1$

$$x_2 = \exp(-0.3 * 1^2) \approx 0.74$$
$$x_3 = \exp(-0.3 * (-2)^2) \approx 0.30$$

More about the RBF Kernel

- The γ parameter is sort of the inverse standard deviation of the Gaussian used to compute the similarity features

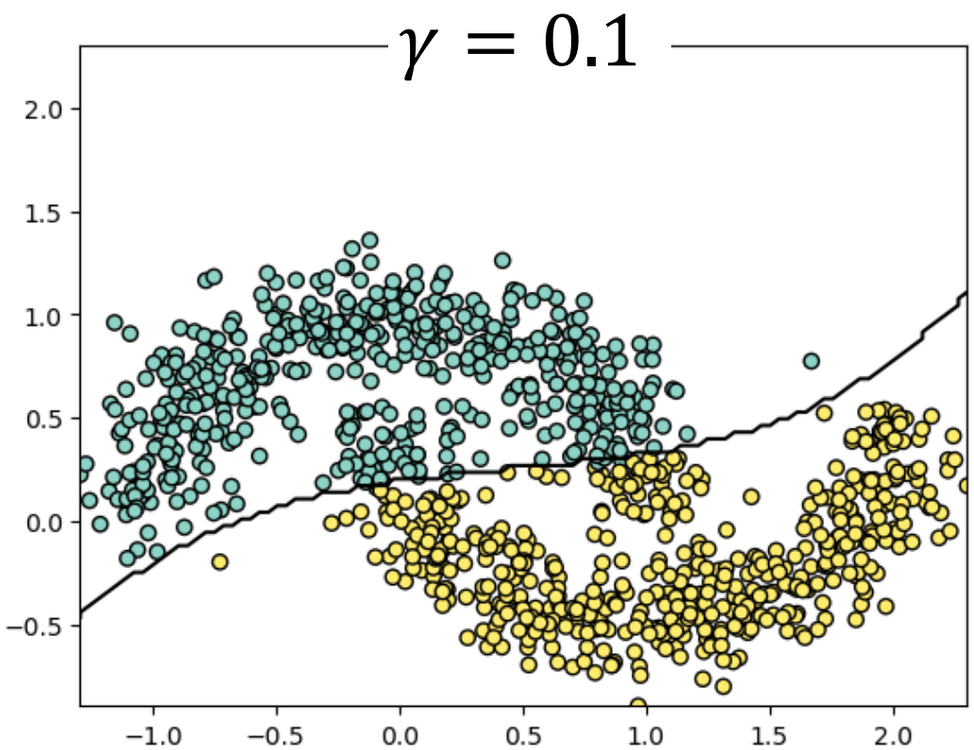
- As γ decreases, the standard deviation gets larger and it is easier to have a non-negligible distance from the landmark

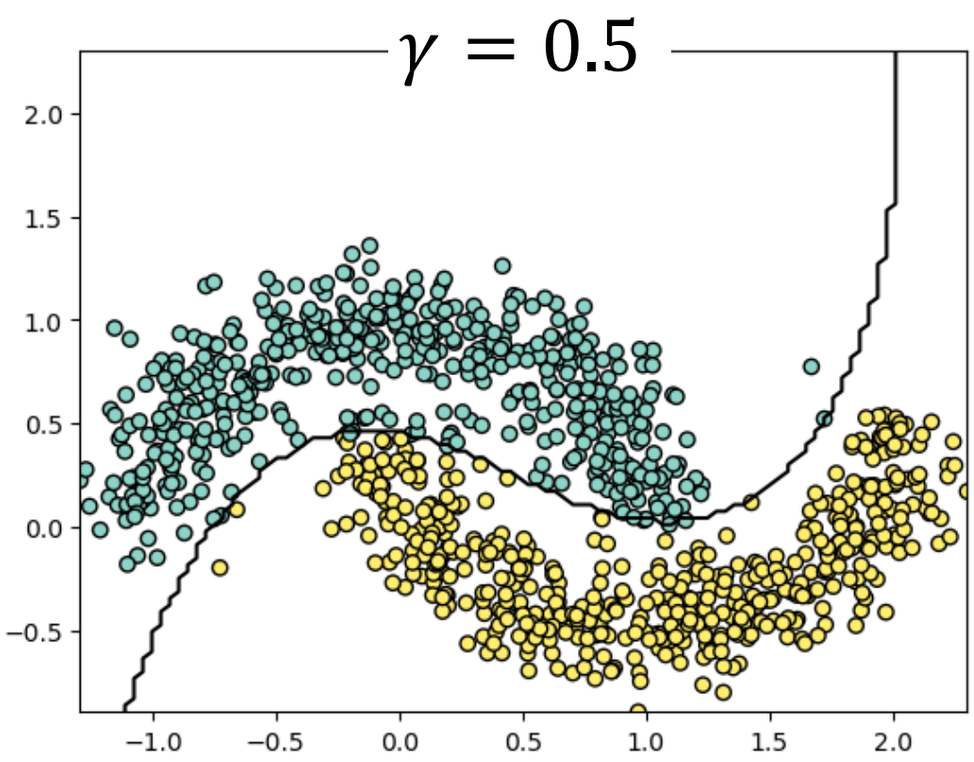
  - Small γ will smooth more and can lead to underfitting

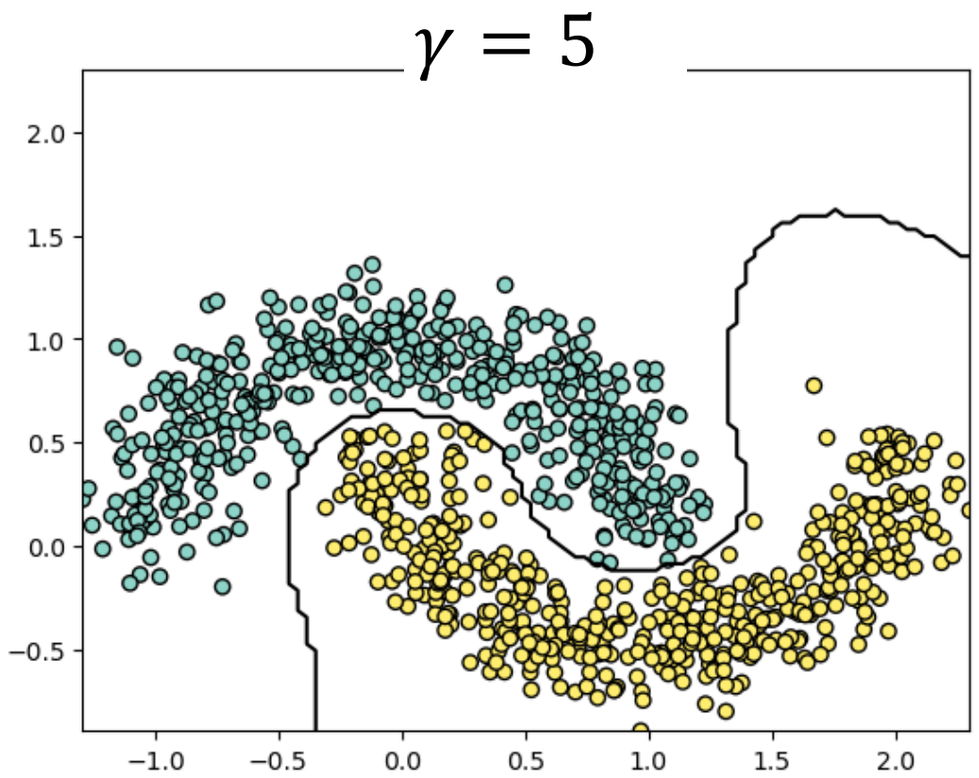
- As γ increases, the standard deviations get smaller and it is many of the distances from the landmark will be very close to zero

  - Large γ will capture small variations and can lead to overfitting

Comparing γ







Gaussian RBF Kernel

- This is a very popular kernel and is the default for the **SVC** function in sklearn

- As γ decreases, the standard deviation gets larger and it is easier to have a non-negligible distance from the landmark

  - Small γ will smooth more and can lead to underfitting

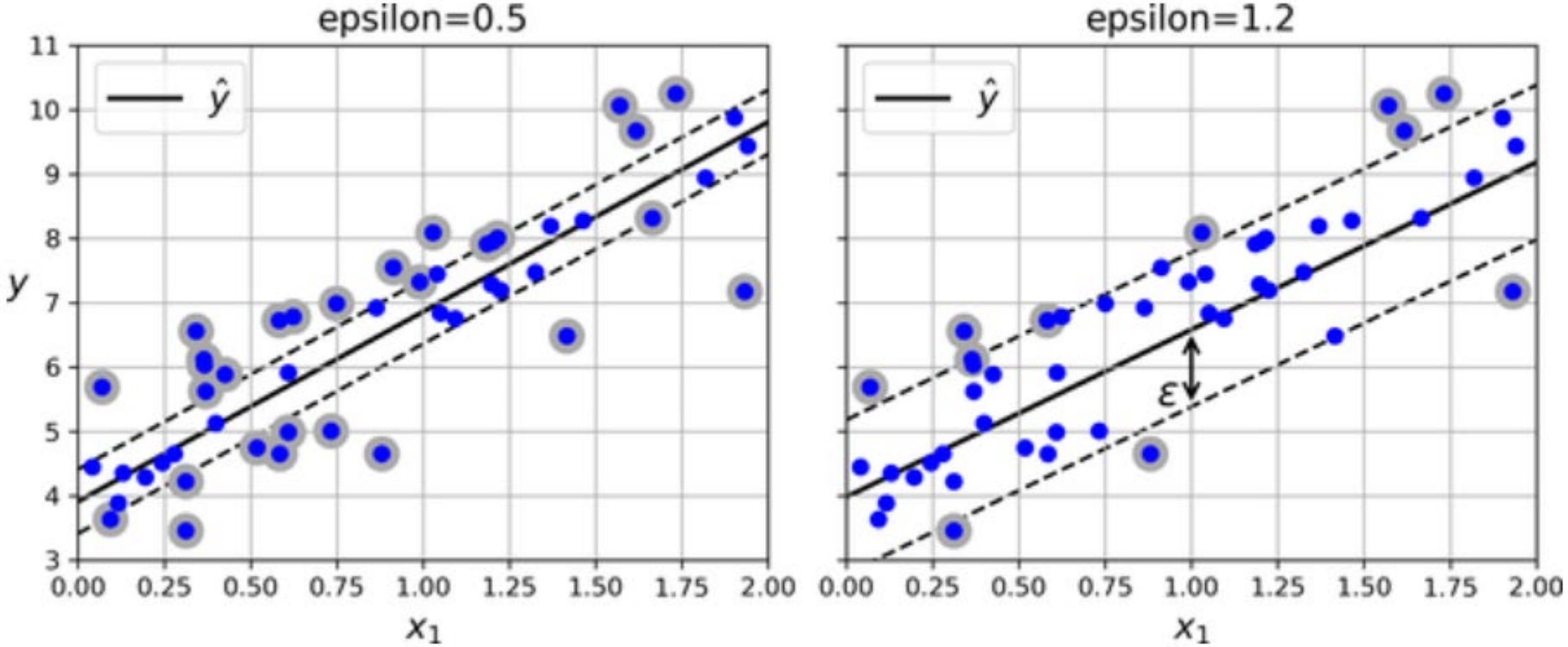
- As γ increases, the standard deviations get smaller and it is many of the distances from the landmark will be very close to zero

  - Large γ will capture small variations and can lead to overfitting

SVM Regression

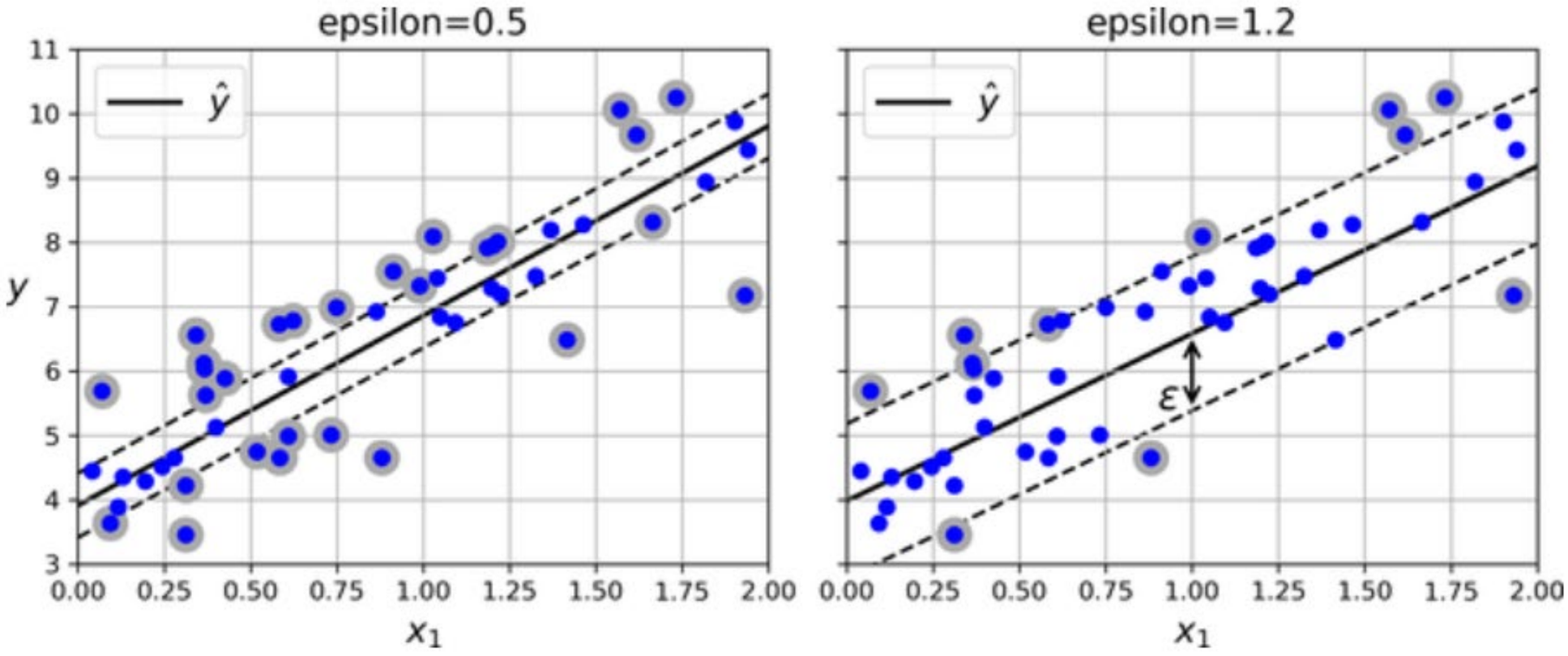
SVM Regression

- SVMs can be used for regression tasks as well
- Here, instead of finding the widest possible road, we want to fit as many observations on the street while limiting the number of points off the street



SVM Regression

- Here, the support vectors are the instances off the street because they are the only points that effect the model's predictions
- If more points are added inside the margin, predictions don't change (this is referred to as being ϵ-insensitive



Summary

- SVMs are a very different way of creating a classifier than other methods that we've looked at
  - No notion of finding $P(y_i|X_i)$
- Only binary classifier
  - Can be extended to multiclass problems using one-versus-one or one-versus-all strategies
- Pretty slow for large datasets